Infrastructure client / serveur dans un contexte industriel

Une cellule robotisée supervisée, de la prise en main du robot à la recette

BACH INGE 2024-0 (3e année) · groupes G1 et G2

5 séances, du 12 octobre au 19 novembre 2026

Le projet fil rouge

Chaque équipe réalise une **cellule robotisée supervisée** et la fait recetter.

Le point de départ : un poste de pick & place du fab lab (robot Niryo Ned2, convoyeur, capteur de présence sur l'entrée DI5) qui fonctionne seul.

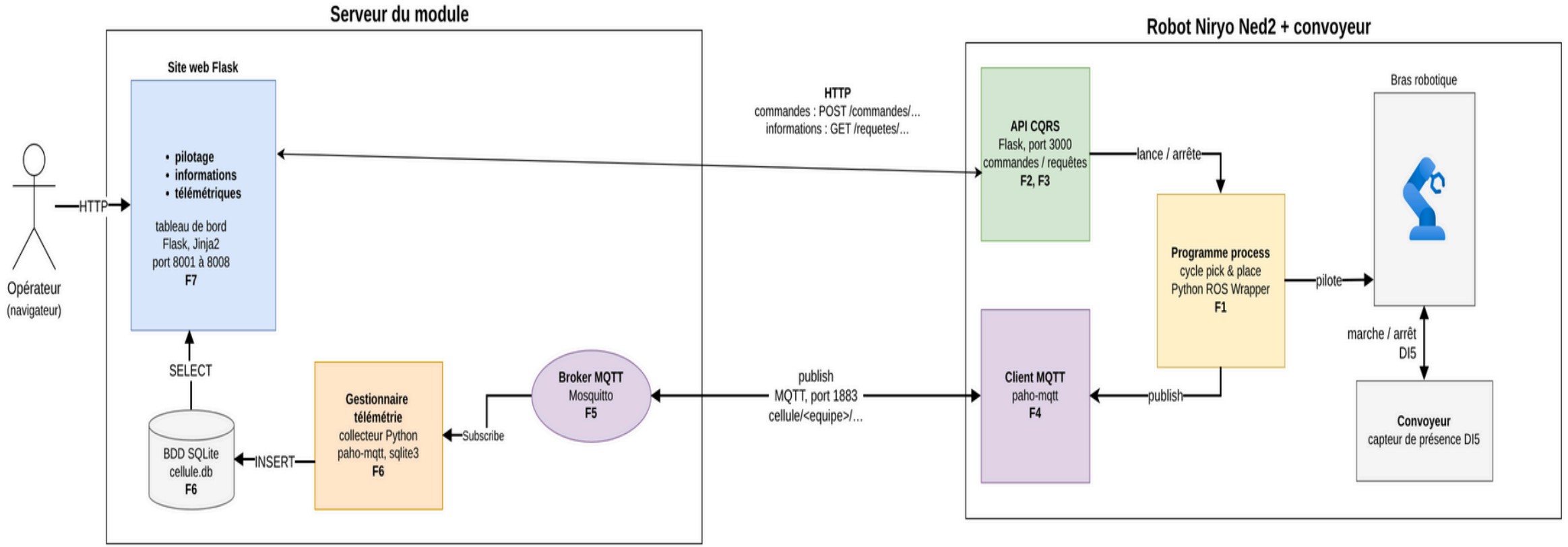
Ce qui manque : piloter à distance, informer, observer la production.

Ce que vous construisez

Robot + convoyeur → **MQTT** → broker Mosquitto → **collecteur Python** → base **SQLite** → **site web Flask** (tableau de bord) → **API de commande** → robot

Vous définissez **votre propre process** : une pièce arrive, le robot la prend, la dépose dans une zone, le cycle est compté et horodaté.

Architecture cible



Fonctions : F1 Exécuter le cycle · F2 Recevoir les commandes · F3 Exposer l'état · F4 Publier les événements · F5 Transmettre les messages · F6 Collecter et stocker · F7 Présenter

Ce que vous saurez faire à la fin

Compétence	
C1	Programmer une séquence robotisée (Blockly puis Python)
C2	Analyser un process industriel et le spécifier (dossier, recette)
C3	Mettre en œuvre une communication client / serveur (MQTT, HTTP)
C4	Collecter et stocker des données de production (SQL / SQLite)
C5	Développer une interface web et une API en Python (Flask)
C6	Intégrer, tester et valider un système (recette)

Déroulement en 5 séances

Séance	Date	Lieu	Thème
1	lun. 12/10 (G1 matin, G2 après-midi)	Fab lab	Prise en main du robot, export Python, process
2	mer. 14/10, 08:30-12:00	Salle C1.07	Analyse, dossier, recette, poste de développement
3	lun. 09/11, 08:30-12:00	Salle D3.10	Intégration sur le serveur
4	mar. 17/11 (G1 matin, G2 après-midi)	Fab lab	Intégration sur les robots, recette à blanc
5	jeu. 19/11 (G1 13:30, G2 15:15)	Fab lab	Évaluation par la recette

- 3h30 par séance, sauf l'évaluation (1h45 par groupe) : **15h45** au total
- 22 étudiants, **8 équipes de 2 ou 3**, 4 robots : une équipe par robot dans chaque groupe
- Les équipes sont constituées à la séance 1 et restent les mêmes jusqu'à la fin

Le rythme du module

```
12/10        14/10                      09/11      16/11   17/11   19/11
  S1           S2     <-- 4 semaines -->  S3       dossier   S4      S5
robot        analyse    travail sur       serveur    v2     robots  évaluation
             + poste    votre laptop
```

- **Entre les séances 2 et 3** : 4 semaines de travail **autonome** sur laptop, sans serveur
- **Entre les séances 3 et 4** : 8 jours, pour tester le process et l'API en mode simulé
- **Entre les séances 4 et 5** : 2 jours, pour corriger avant l'évaluation

Le robot n'est disponible qu'aux séances 1, 4 et 5 : **préparez tout le reste avant**.

Livrables à chaque étape

Étape	Livrables
Séance 1	Programme Blockly et export `.py` (avec les poses), fiche process v0 (une page)
Séance 2	Dossier d'analyse v1, cahier de recette v1 (8 tests au moins), présentation flash, poste validé
Entre 2 et 3	Chaîne complète en local : simulateur, collecteur, base SQLite, site web, API simulée
Séance 3	Collecteur, site web et API (mode simulé) déployés sur le serveur
Avant la séance 4	Dossier v2 (lun. 16/11), programme process et API testés en mode simulé
Séance 4	Système intégré sur le robot, cahier de recette pré-rempli
Séance 5	Recette signée, code déposé, dossier final

Ce qui vous est fourni

Dossier de départ remis à la séance 2

Documents à remplir	Briques logicielles
Gabarit et exemple de **dossier d'analyse**	**Howto** du poste de développement + `verifier_poste.py`
Exemple de **cahier de recette** (3 tests rédigés)	**Simulateur** de process et de robot (`RobotSimule`)
Exemple de **présentation flash**	**Collecteur MQTT** à configurer
	Site web Flask de base

Aussi fournis

- Cours en **libre service** : MQTT, SQL, Python et HTTP, HTML / CSS, Flask, API CQRS, réseau TCP/IP
- Un **compte par équipe** sur le serveur (séance 3), le **broker** et le serveur Raspberry Pi
- Les robots, convoyeurs et pièces (séances 1, 4 et 5), avec Flask et paho-mqtt **déjà installés** sur chaque robot

Connaissances nécessaires à chaque étape

Étape	Vous devez savoir	Support
Séance 1	Bases de Python (1re année), consignes de sécurité du robot	`seance1`, `README.md`
Séance 2	Notion de client / serveur, format JSON, plan d'un dossier	`README.md`
Entre 2 et 3	Publier et recevoir en MQTT, `CREATE` / `INSERT` / `SELECT`, routes Flask	`collecteur_MQTT`, `SQL`, `intro_flask`
Entre 2 et 3	HTML / CSS de base, requêtes HTTP `GET` / `POST`	`site_web_HTML_CSS`, `API_CQRS_REST_Flask`
Séance 3	SSH, adresse IP et ports	`reseau_TCP_IP`
Séance 4	Réseau de la cellule, passage de l'API simulée au robot	`reseau_TCP_IP`, `API_CQRS_REST_Flask`
Séance 5	Dérouler un test de recette, expliquer son architecture et son code	Votre dossier

Scénario de tâches

Ce que chaque équipe fait, séance après séance

Séance 1 : prise en main du robot

Lieu : fab lab · **Durée** : 3h30 · **Matériel** : robot Ned2 et convoyeur

Durée	Tâche à réaliser
20 min	Constituer l'équipe, suivre les consignes de sécurité (zone, arrêt d'urgence, vitesses)
60 min	Connecter Niryo Studio, calibrer, utiliser le mode apprentissage, la pince
70 min	**Scénario guidé** : convoyeur jusqu'au capteur DI5, prise de la pièce, dépose, retour en attente
30 min	Exporter en Python, repérer `calibrate_auto`, `move_pose`, `pick_from_pose`, `control_conveyor`, `digital_read`
30 min	Décrire le process : étapes, entrées / sorties, cas d'erreur, premières données à remonter

À déposer : programme Blockly et `.py` (avec les poses, à conserver pour la séance 4), fiche process v0

Support : `cours/seance1.md` (points à réaliser, fonctions du robot, fiche des poses)

Séance 2 : analyse, dossier, poste de développement

Lieu : salle C1.07 · **Durée** : 3h30 · **Matériel** : votre laptop

Durée	Tâche à réaliser
30 min	Cours : architecture client / serveur, fonctions F1 à F7, cahier des charges
30 min	Observer la chaîne fournie : simulateur → broker → collecteur → SQLite → site
25 min	Installer le poste avec le howto et lancer `verifier_poste.py`
70 min	Rédiger le **dossier d'analyse** et le **cahier de recette** à partir des exemples
40 min	**Présentation flash** : 5 min par équipe
15 min	Répartir le travail et fixer les jalons jusqu'à la séance 3

À déposer : dossier v1, recette v1 (8 tests au moins), présentation, poste validé

Entre les séances 2 et 3 : 4 semaines en autonomie

Semaine	Tâches	Cours à lire
1	Poste validé, fournitures lancées en local, **topics et message JSON** définis	`collecteur_MQTT`
2	**Simulateur** adapté à vos données, **collecteur** configuré, tables SQLite créées	`SQL`, `collecteur_MQTT`
3	**Site web** adapté : dernier état, liste des cycles, compteurs	`site_web_HTML_CSS`, `intro_flask`
4	**API de commande** en mode simulé reliée au site, dossier complété	`API_CQRS_REST_Flask`

- Broker : Mosquitto en local, ou un broker public de test (topics préfixés par un identifiant propre à l'équipe, aucune donnée sensible)
- Vous arrivez à la séance 3 avec une **chaîne qui fonctionne en local**

Séance 3 : intégration sur le serveur

Lieu : salle D3.10 · **Durée** : 3h30 · **Matériel** : serveur Raspberry Pi, laptops

Durée	Tâche à réaliser
40 min	**Démonstration** de votre chaîne en local (5 min par équipe)
20 min	Recevoir vos codes d'accès, se connecter au serveur (SSH)
30 min	Passer du broker local au **broker du serveur** (fichier de configuration)
40 min	Déployer le **collecteur** et la base SQLite, vérifier par des requêtes (nombre de cycles, durée moyenne)
40 min	Déployer l'**API** simulée (port 5001 à 5008)
40 min	Déployer le **site web** (port 8001 à 8008) avec ses boutons de commande

À déposer : collecteur, base, API et site fonctionnels sur le serveur

Entre les séances 3 et 4 : 8 jours

- Terminer le **dossier v2** : topics, format des messages, schéma de la base, routes de l'API (**rendu lun. 16/11**)
- Préparer dans le programme du robot les **appels de publication MQTT**
- Tester le programme process et l'API **en mode simulé** : seul le passage au robot réel doit rester à faire
- Relire l'export Python de la séance 1 et les poses relevées

> Les étudiants n'ont le robot que 3h30 à la séance 4. Tout ce qui n'est pas prêt avant ne sera pas terminé.

Séance 4 : intégration sur les robots

Lieu : fab lab · **Durée** : 3h30 · **Matériel** : robot, convoyeur, serveur

Durée	Tâche à réaliser
15 min	Vérifier le réseau : IP du robot et du serveur, accès du robot au broker, accès du serveur à l'API (port 3000)
30 min	**Reprise en main** : sécurité, calibration, relecture des poses
1h30	**Déployer** sur le robot le programme process, la publication MQTT et l'API (`RobotNiryo`) : copie par `scp` , lancement par `~/cellule/lancer.sh`
45 min	**Essai de bout en bout** (robot → serveur → site web → robot) et recette à blanc
30 min	**Bilan** : corrections avant l'évaluation, mise à jour du cahier de recette

À déposer : système intégré, cahier de recette pré-rempli

Séance 5 : évaluation par la recette

Lieu : fab lab · **Durée** : 1h45 par groupe, environ 25 min par équipe

Pendant qu'une équipe passe, la suivante prépare sa cellule sur un autre robot.

Durée	Déroulé
18 min	**Recette** : vous déroulez les tests de votre cahier choisis par l'enseignant, chaque test est conforme ou non conforme
7 min	**Questions individuelles** sur l'architecture et le code

Proposition de grille (à valider)

Élément	Poids
Dossier d'analyse et cahier de recette (version finale)	30 %
Recette : tests conformes, fonctionnement de bout en bout	50 %
Questions individuelles	20 %

Les points de vigilance

- **Séance 2 courte** : appuyez-vous sur les exemples fournis, terminez le dossier dans la semaine si besoin
- **Quatre semaines sans encadrement** : suivez les jalons semaine par semaine
- **Séance 3** : sans démonstration locale, vous repartez des fournitures de la séance 2
- **Temps robot limité** (séances 1, 4 et 5) : un process simple, bien testé en simulé
- **Fin de module serrée** : séance 3 le 09/11, séance 4 le 17/11, évaluation le 19/11

Chaque équipe déroule son propre cahier de recette : écrivez des tests que vous saurez réussir et expliquer.

Questions ?

Supports : `README.md` et dossier `cours/`

Progression détaillée : `progression_pedagogique/BACHINGE24.md`